
What is AWS Step Functions?

AWS Step Functions is a **serverless orchestration service** that allows you to coordinate **multiple AWS services** into **serverless workflows**, using a **visual workflow editor**.

 Think of it like a **flowchart** that defines how different AWS services and components interact — with logic like branching, retries, and parallelism.

Key Concepts & Components

Concept	Description
 State Machine	A JSON-based workflow definition using the Amazon States Language (ASL). It contains a series of states that represent tasks, logic, and transitions.
 State	A step in the workflow. Can be a task, choice, parallel step, wait, etc.
 Task State	A single unit of work. Can invoke Lambda, ECS, Fargate, EMR, Glue, etc.
 Choice State	A branching decision, like an if/else condition.
 Wait State	Pauses the execution for a set time or until a timestamp.
 Parallel State	Runs branches of states concurrently .
 Success/Fail States	Explicitly end execution successfully or with failure.
 Execution	A single run of the state machine with its own input/output.



Execution History

Full log of every step's inputs/outputs and transitions.



Amazon States Language (ASL)

A JSON-based language to define workflows.



Step Functions Types

1. Standard Workflows

- Long-running (up to 1 year)
- Good for complex processes (ETL, human approvals)
- Includes detailed execution history

2. Express Workflows

- Short-duration (less than 5 minutes)
 - High throughput (100,000+ per second)
 - Ideal for high-volume event-driven workflows (IoT, streaming, APIs)
-



Common Integrations

Step Functions can natively integrate with:

-  **AWS Lambda** — run serverless compute functions

-  **Amazon SNS / SQS** — send messages or trigger downstream services
 -  **AWS Glue** — orchestrate ETL jobs
 -  **Amazon EMR** — process big data workflows
 -  **Amazon SageMaker** — automate ML training/prediction
 -  **Amazon ECS / Fargate** — run containers
 -  **API Gateway** — trigger state machines from APIs
 -  **DynamoDB, S3, Athena, Redshift, etc.**
-

Sample Workflow Diagram

```
{
  "StartAt": "GetOrder",
  "States": {
    "GetOrder": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:....:function:GetOrder",
      "Next": "CheckInventory"
    },
    "CheckInventory": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.stock",
          "NumericGreaterThan": 0,
          "Next": "ShipOrder"
        }
      ]
    }
  }
}
```

```

    ],
    "Default": "NotifyOutOfStock"
  },
  "ShipOrder": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:....function:ShipOrder",
    "End": true
  },
  "NotifyOutOfStock": {
    "Type": "Fail",
    "Cause": "No inventory"
  }
}
}
}

```

Use Cases

Use Case	Description
 Data Processing Pipelines	Chain ETL steps using Glue, Lambda, EMR
 Approval Workflows	Include human decision steps (via SNS + Lambda)
 Microservice Orchestration	Coordinate multiple service calls reliably
 Retry Logic	Automatically retry failed steps with exponential backoff
 API Backend Logic	Create stateful backend logic for APIs via API Gateway

Benefits

-  **Modular:** Break down logic into reusable tasks
-  **Scalable:** Handles thousands of executions concurrently
-  **Auditable:** Execution history is fully visible in the console
-  **Low-Code:** Visual editor + JSON definitions
-  **Cost-efficient:** Pay per transition in Standard, or by invocation in Express

Teaching Notes (with Emojis)

Emoji	Topic	Example
	State Machine	Flow of business logic (like a program)
	Choice State	Like if/else or switch
	Parallel State	Run multiple steps at the same time
	Task State	Calls Lambda or another service
	End State	Marks end of a path
	Wait State	Delay steps (e.g., for retry or polling)
	Fail State	Explicit failure of flow
	Execution History	Logs of every step's inputs and outputs

Great! Here's a **Step-by-Step Lab Document** for the "**Serverless Order Processing Flow**" using **AWS Step Functions**, **Lambda**, **DynamoDB**, and **SNS**. Ideal for students learning how to coordinate AWS services using a visual, no-server approach.

AWS Lab: Serverless Order Processing Workflow

Objective:

Design a serverless order-processing workflow using **AWS Step Functions**, triggered via **API Gateway**, and orchestrating **Lambda**, **DynamoDB**, and **SNS**.

Architecture Diagram:

API Gateway

|



Lambda: Validate Order

|



Lambda: Check Stock → DynamoDB

|



Choice State:

├── Stock Available: Lambda: Ship Order

└── Stock Unavailable: SNS: Out-of-Stock Notification

AWS Services Used:

Service	Purpose
API Gateway	Triggers the workflow via HTTP
Lambda	Validates orders, checks stock, ships
DynamoDB	Stores item stock
Step Functions	Orchestrates the workflow
SNS	Sends out-of-stock notifications

Lab Steps:

Step 1: Create DynamoDB Table

- Table Name: **ProductInventory**
- Partition Key: **productId** (String)
- Sample item:

```
{  
  "productId": "123",  
  "stock": 5  
}
```

Step 2: Create Lambda Functions

Lambda: **ValidateOrderFunction**

- Validates order input (e.g., check productId exists)
- Returns the validated order JSON

```
def lambda_handler(event, context):  
    if "productId" not in event:  
        raise ValueError("Missing productId")  
    return event
```

2 Lambda: **CheckStockFunction**

- Reads stock from DynamoDB and adds stock count to the state

```
import boto3  
  
dynamodb = boto3.resource('dynamodb')  
table = dynamodb.Table('ProductInventory')  
  
def lambda_handler(event, context):  
    product_id = event['productId']  
    response = table.get_item(Key={'productId': product_id})  
    item = response.get('Item', {})  
    stock = item.get('stock', 0)  
    event['stock'] = stock  
    return event
```

3 Lambda: **ShipOrderFunction**

- Dummy shipping logic (could be extended)

```
def lambda_handler(event, context):
    return {
        "message": f"Order for product {event['productId']} shipped."
    }
```

✅ Step 3: Create SNS Topic

- Topic name: **OutOfStockTopic**
 - Add an email subscription for testing
 - Note down the ARN for Step Functions
-

✅ Step 4: Create Step Function Workflow

Use the following Amazon States Language (ASL):

```
{
  "StartAt": "Validate Order",
  "States": {
    "Validate Order": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:REGION:ACCOUNT:function:ValidateOrderFunction",
      "Next": "Check Stock"
    },
    "Check Stock": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:REGION:ACCOUNT:function:CheckStockFunction",
      "Next": "Is Stock Available?"
    },
    "Is Stock Available?": {
      "Type": "Choice",
```

```

    "Choices": [
      {
        "Variable": "$.stock",
        "NumericGreaterThan": 0,
        "Next": "Ship Order"
      }
    ],
    "Default": "Notify Out of Stock"
  },
  "Ship Order": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:REGION:ACCOUNT:function:ShipOrderFunction",
    "End": true
  },
  "Notify Out of Stock": {
    "Type": "Task",
    "Resource": "arn:aws:states:::sns:publish",
    "Parameters": {
      "Message": "Product out of stock: No stock available for productId = $.productId",
      "TopicArn": "arn:aws:sns:REGION:ACCOUNT:OutOfStockTopic"
    },
    "End": true
  }
}

```

Replace **REGION** and **ACCOUNT** with your AWS region and account ID.

Step 5: Trigger via API Gateway (Optional)

- Create an API Gateway endpoint that triggers the Step Function using AWS SDK or Lambda proxy
- Alternatively, trigger the Step Function execution manually with test JSON:

```
{  
  "productId": "123"  
}
```

Testing

Scenario	Input	Expected Result
 In Stock	<code>{ "productId": "123" }</code>	Shipping Lambda is called
 Out of Stock	<code>{ "productId": "999" }</code> (or stock=0)	SNS notification sent

Cleanup

- Delete Step Function
 - Remove Lambda functions
 - Delete DynamoDB table
 - Remove SNS topic/subscription
-

